# AI Applications in Research

Agentic Empirical Work and the Use of Claude Code

Prepared from *Scott's Mixtape Substack* and related sources

August 2026

**Abstract**

This document summarises the main concepts governing the use of AI coding agents—especially **Claude Code**—in contemporary empirical research. It is synthesised primarily from Scott Cunningham's mid-2026 writing on agentic causal-inference work. The central claim is that AI agents have collapsed the marginal cost of *producing* research to near zero, which shifts the binding constraint away from production and toward **verification** and, most acutely, **human understanding**. The document lays out the production–verification–understanding framework, the discipline of research "harnesses" and checklists, concrete Claude Code workflows (skills, hooks, the git-diff unit of review, and quizzing), and the open risks of automating cognitive work.

## 1 The core shift: production is cheap now

By mid-2026 it is no longer a conjecture that AI agents can autonomously produce quantitative empirical research. A coding agent such as Claude Code or Codex can, from a single prompt, choose a research question, pick an identification strategy, locate and download datasets, select estimators, write replicable code, produce tables and figures with robustness checks, and draft a full 20-page manuscript with references and appendices—often in under half an hour. Cunningham calls this "3-d printing a research paper."

The economic framing is a leftward-and-downward collapse in the marginal cost of research production, from $MC_b$ (before AI) to $MC_{AI} \approx \varepsilon > 0$:

$$MC_b \longrightarrow MC_{AI} \approx \varepsilon, \qquad Q_b^* \longrightarrow Q_{AI}^* \gg Q_b^*. \tag{1}$$

Output rises to the point where marginal benefit equals the new (tiny) marginal cost. Crucially, cheap production does *not* imply that the agent is a fully autonomous researcher: the current state is closer to lane-keeping plus cruise-control than to full self-driving. The human remains responsible for choosing where to go and for certifying that the destination was actually reached.

## 2 The production–verification–understanding framework

Cunningham's organising idea is that any research manuscript must pass through *three distinct doors*. Before AI, all three were bundled together because the act of producing the work produced verification and understanding almost as by-products. Agents unbundle them.

**Production**

The generation of code, exhibits, and prose. This is the stage agents now do well and can do autonomously. Evidence: the Social Catalyst Lab produced roughly 1,000 automated program-evaluation papers where humans controlled the prompts but agents controlled the production.

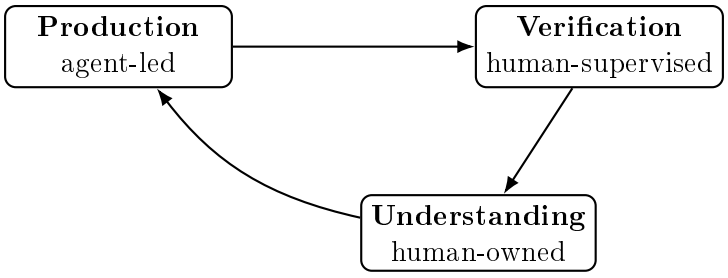



Figure 1: The three doors every AI-assisted manuscript must pass through.

**Verification**
Confirming the accuracy of what was produced. Replicating code across languages or checking that every figure traces to stored code are *necessary but not sufficient*. Verification is defined *relative to a pre-registered target estimand*: it means confirming that what should have been done was done *and that what should not have been done was not done*. The "was-not-done" half cannot be delegated to the agent, because there is nothing for the agent to point at. Agents can assist verification but should not be its sole authority.

**Understanding**
The researcher's own comprehension of the work. This is the hardest door and, increasingly, the true bottleneck. The analogy is the CEO who need not write the code but must know, approximately and correctly, what was done, by whom, and what was found.

### 2.1 "Verification is the bottleneck" vs. "understanding is the bottleneck"

A popular slogan holds that *verification* is the new bottleneck. Cunningham argues the sharper statement is that **understanding** is the new bottleneck: agents may eventually verify as well as they produce, yet that still would not guarantee that the human understands the result. Under the old mode, understanding was almost an *externality* of production—a by-product created by the act of doing the work. Automating production removes that by-product, and it is an open empirical question whether the time freed up can be reinvested to reproduce the lost understanding.

### 2.2 Zero errors as a constraint, not an objective

A recurring principle: **zero errors is a constraint, not the objective function**. Framing the goal as "minimise errors" quietly licenses the notion of "optimal errors," which is unacceptable when the supply elasticity of research with respect to price is large. Formally, optimal research output maximises net social benefit *subject to* the zero-error constraint:

$$\max_{\text{output}} \big[\text{social benefit} - \text{social cost}\big] \quad \text{s.t.} \quad \text{errors} = 0,\ \text{understanding} \geq \bar{u}. \tag{2}$$

## 3 Harnesses and checklists: disciplining the agent

To keep agentic research inside the three doors, Cunningham builds a **research harness**—in his case a **difference-in-differences harness** that is really a staged checklist, evolving toward a piece of software. The checklist idea is borrowed from Atul Gawande's *The Checklist Manifesto*: experts skip steps more than novices (Kahneman's fast "System 1" thinking), so high-stakes work needs explicit checklists. The harness is deliberately *narrow* (diff-in-diff only) so it can be evaluated and

perfected, though the same structure extends to synthetic control, unconfoundedness, and other designs.

### 3.1 Selected checklist stages

**Packages**
A visual card fixing which packages and versions the project will use, pinned for reproducibility.

**Bite**
The heart of the design work: document *the very first thing an intervention does* in the real world before looking at ultimate outcomes. If a gun law passes, study the movement of guns, not deaths; if the minimum wage rises, find where it binds. Be exhaustive across two dimensions—*space* (heat maps of first-stage response) and *time* (time-series aggregations, including comparison groups). Do not exit "bite" until every helpful dataset has been found and documented.

The bite stage is where insight into the **treatment assignment mechanism** develops—the design-based tradition (Angrist, Imbens, Rubin) emphasises understanding *how* treatment was assigned rather than parametrically modelling outcomes.

### 3.2 Treatment assignment mechanisms

The harness ties each design to the mechanism that assigns treatment:

- **Human rationality / essential heterogeneity** (endogenous choice; Heckman–Roy): the case that makes naive causal inference nearly impossible and is the reason design matters (LaLonde 1986).
- **Instruments** that assign treatment to compliers only ⇒ instrumental variables and LATE (Imbens & Angrist 1994); randomization is buried inside the instrument, subject to the exclusion restriction.
- **Randomized treatments** acting directly on the outcome ⇒ RCTs (randomization on the treatment itself).
- **Running variables** crossing a threshold (e.g. blood-alcohol $> 0.08$ ⇒ DUI) ⇒ regression discontinuity (Imbens & Lemieux guide to practice).

### 3.3 What to automate—and what not to

A key judgement: *do not fully automate the bite stage.* Agents should make the maps, write the code, produce the time series, and download and organise the data—the accuracy-oriented tasks that feed verification—but the human must own the conception of the project and the reading of the treatment assignment mechanism. In diff-in-diff, bite is used only to *understand* and to justify control-group choices (unlike IV, where the first stage also enters estimation). Automating the parts that generate understanding is precisely where value is lost.

## 4 Concrete Claude Code workflows

**Skills**
Reusable, invokable routines (e.g. a `/quiz` skill). Skills package a repeatable task so the agent performs it consistently.

**Hooks**
Automation attached to specific checklist steps—for example, hooks that standardise figure and table production and wire outputs into a deck or manuscript. The guidance is to make hooks *specific to each checklist stage* rather than abstract and universal.

**The git-diff unit of review**
Following Paul Goldsmith-Pinkham, the "unit of verification is the git diff." Reviewing each diff is meant to clear the "cognitive debt" of not having written the code. The honest caveat: it is easy to *skim and accept* diffs, silently accumulating debt rather than clearing it—so the diff review is necessary but not sufficient for understanding.

**The `/quiz` skill**
An antidote to passive diff-acceptance. Claude Code (i) generates slides that teach the researcher the current piece of the project, then (ii) asks multiple-choice questions one at a time—modelled on corporate "training-video-plus-quiz" compliance. The goal is to manufacture *continuous understanding* on top of verification and the staged checklist.

**Interviews for elicitation**
Structured agent interviews extract the researcher's priors—e.g. which covariates to include and what the target parameter is—in cases where the belief is latent but real.

## 5 Risks, tensions, and open questions

- **Principal–agent / skill atrophy.** Trusting an agent whose work you do not check can become "the blind leading the blind"; if skills atrophy, the researcher may lose even the ability to recognise good work.
- **Endogenous specification search.** Experiments show agents can drift into specifications in response to *true statements* made in a project's context—not just explicit instructions—which complicates trust in agent self-verification.
- **Cognitive debt.** Mechanical diff-acceptance can leave understanding permanently unpaid.
- **Model heterogeneity.** Frontier models differ in their fitness for autonomous research, and whether that heterogeneity is knowable *ex ante* is unresolved.
- **The two corner solutions** (after IBM via Goldsmith-Pinkham): because machines cannot be held accountable, either they help the human understand the work, or they are not used to make decisions at all.

## 6 Practical takeaways

1. Treat yourself as the **CEO** of the research: delegate production, but own verification and understanding.
2. Build a **narrow, staged harness** (start with one design) rather than a general-purpose prompt.
3. Fix a **target estimand first**; verify against it, checking both what was done and what should not have been done.
4. **Do not automate conception** (the "bite" / assignment-mechanism work) even while automating exhibits.

5. Pair **git-diff review** with an active understanding check such as a **quiz** to avoid cognitive debt.

6. Hold **zero errors as a constraint**; never slide into "optimal errors."

## Sources

Synthesised from *Scott's Mixtape Substack* (https://causalinf.substack.com), July–August 2026, including "Which stages of research production should we let AI Agents loose" (14 Aug 2026), "How to improve understanding with Claude Code" (7 Aug 2026), "My strategy for talks on agents for research" (10 Aug 2026), and "Online version of The Remix is now here" (17 Aug 2026); with reference to G. Litt, "Understanding is the new bottleneck," and P. Goldsmith-Pinkham's notes on git-diff verification.